

Energy Efficiency of Locally-Hosted Open-Source LLMs on Automated Software Engineering Tasks

Anonymous Author(s)

Abstract

Large language models (LLMs) are increasingly deployed as autonomous coding agents, yet little is known about their energy footprint when running on local GPU hardware. We benchmark three open-source LLMs of comparable size—Mistral Small 24B, Gemma 3 27B, and Qwen 3.6 27B—on the SWE-bench Lite dataset using the OpenHands agent framework, measuring per-second GPU power draw on a single NVIDIA A100 80 GB PCIe accelerator. Qwen 3.6 27B resolves 51 of 87 evaluated instances (58.6%), compared to 10/65 (15.4%) for Mistral and 8/97 (8.2%) for Gemma. Despite consuming 12,150 Wh of total GPU energy—roughly three times more than Mistral—Qwen achieves the lowest energy cost per resolved instance (238 Wh vs. 421 Wh). We find that all bugs resolved by Mistral are a strict subset of those resolved by Qwen, and that 39 bugs are resolved exclusively by Qwen. Analysis of tool-call patterns reveals distinct problem-solving strategies: Mistral favors file editing (68% of tool calls), while Qwen relies predominantly on terminal commands (55%). Our results demonstrate that raw energy consumption is a misleading proxy for efficiency, and that task-effectiveness-normalized metrics are necessary for meaningful comparisons.

1 Introduction

The rapid adoption of LLM-based coding agents [?] raises questions about their computational and environmental cost. While API-hosted models amortize inference across data centers with optimized batching and hardware utilization, an increasingly common deployment mode is *local hosting*: running open-weight models on dedicated GPU hardware for reasons of data privacy, latency, or cost control.

In this setting, the GPU is exclusively allocated to a single model, and its power draw—typically 200 W to 300 W for a data-center accelerator—is borne entirely by the user. Unlike API pricing, which abstracts energy into token costs, local hosting exposes the physical energy budget directly.

We present an empirical study of three open-source LLMs, each in the 24–27 B parameter range, running on identical hardware (a single NVIDIA A100 80 GB PCIe

GPU) against the SWE-bench Lite benchmark [?]. Our contributions are:

1. A controlled comparison of task-completion effectiveness and GPU energy consumption across three models under identical conditions.
2. Per-second GPU power telemetry (153,659+ samples per run) enabling fine-grained energy attribution to individual task instances.
3. Evidence that total energy consumption is a poor proxy for efficiency—the model consuming the most energy per run is also the most efficient per resolved bug.
4. Analysis of agent behavioral differences (tool-usage patterns, iteration depth) and their correlation with both effectiveness and energy.

2 Related Work

LLM-based coding agents. SWE-bench [?] introduced a benchmark of real-world GitHub issues for evaluating autonomous coding agents. OpenHands [?] provides an open-source agent framework supporting multiple LLM backends. Prior evaluations focus on resolve rates and token efficiency; energy consumption has not been systematically measured.

Energy measurement of ML workloads. GPU energy measurement using NVML power queries has been validated as accurate to within 5% of wall-outlet measurements [?]. Prior work has quantified training energy [?], but inference energy for agentic workloads—where a single task may involve dozens of LLM calls interspersed with tool execution—remains under-explored.

3 Methodology

3.1 Hardware and Software

All experiments run on a single Azure Standard_NC24ads_A100_v4 virtual machine equipped with one NVIDIA A100 80 GB PCIe GPU (TDP 300 W), 24 vCPUs, and 220 GB RAM. The software stack comprises vLLM 0.25.1 for model serving, OpenHands SDK v1.16.1 as the agent framework, and Docker containers for per-instance sandboxed evaluation.

Table 1: Models under evaluation. All served via vLLM with 32,768-token context and no native tool calling.

Model	Params	dtype	tok/s (gen)
Mistral Small 24B	24B	fp16	25.8
Gemma 3 27B	27B	bf16	17.9
Qwen 3.6 27B	27B	bf16	23.5

3.2 Models

We evaluate three instruction-tuned models of comparable size, summarized in Table 1:

3.3 Benchmark

We use SWE-bench Lite, a curated subset of 300 instances from the full SWE-bench dataset. Each instance is a real GitHub issue paired with a test patch; the agent must produce a code patch that passes the held-out tests. We attempt 100 instances per model (`n_limit=100`) with a maximum of 100 agent iterations per instance.

3.4 Energy Measurement

GPU power is sampled at 1 Hz via `nvidia-smi` throughout each run, yielding a continuous power trace. Total energy for a phase p is computed as:

$$E_p = \sum_{t \in p} P(t) \cdot \Delta t \quad (1)$$

where $P(t)$ is the instantaneous GPU power at time t and $\Delta t = 1$ s.

An idle baseline measurement is taken before each run (GPU loaded with model weights but no active inference). Baseline power ranges from 43 W to 45 W across runs. Net energy is computed by subtracting the idle baseline.

Per-instance energy is derived from the power trace using each instance’s start and end timestamps:

$$E_i = \bar{P}_i \cdot d_i / 3600 \quad (2)$$

where $\bar{P}_i$ is the mean GPU power during instance i and d_i is the duration in seconds.

4 Results

4.1 Task Completion

Table 2 summarizes the fate of 100 attempted instances per model. Figure 1 visualizes the outcome distribution.

Qwen 3.6 27B substantially outperforms both other models, resolving 51 of 87 evaluated instances (58.6%). Mistral resolves 10/65 (15.4%) and Gemma 8/97 (8.2%).

The models exhibit distinct failure modes. Mistral times out on 35 of 100 instances without producing any

Table 2: Instance outcomes. Each model attempted 100 SWE-bench Lite instances.

	Mistral	Gemma	Qwen
Attempted	100	100	100
Timed out	35	3	13
Empty patch	1	15	0
Evaluated	65	97	87
Resolved	10 (15.4%)	8 (8.2%)	51 (58.6%)

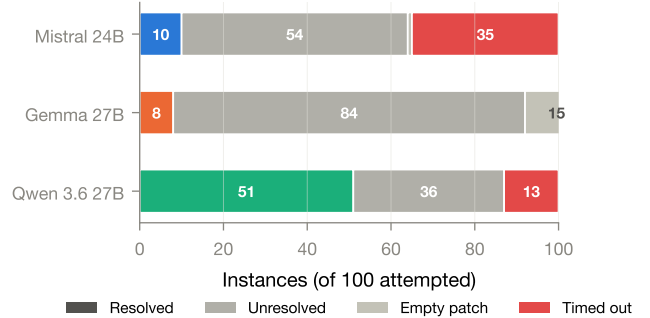


Figure 1: Instance outcome breakdown per model. The resolved segment (colored) uses each model’s identifying color; gray shades indicate unresolved and empty-patch instances; red indicates timeouts. Mistral loses 35% of instances to timeouts before producing any patch, while Gemma produces output for 97 instances but 15 patches are empty.

patch, suggesting it frequently enters unproductive loops. Gemma rarely times out (3/100) but produces 15 empty patches, indicating it reaches the `finish` action without making code changes. Qwen has 13 timeouts and zero empty patches.

4.2 Resolved Instance Overlap

Figure 2 shows the overlap of resolved instances across models. Notably:

- All 10 bugs resolved by Mistral are also resolved by Qwen—Mistral’s resolved set is a strict subset.
- 39 bugs are resolved exclusively by Qwen.
- Only 5 bugs are resolved by all three models.
- 1 bug (`django-15061`) is resolved by Gemma but not by Qwen, suggesting complementary strengths in rare cases.

4.3 Energy Consumption

Table 3 presents the energy measurements and Figure 3 visualizes the comparison.

Qwen consumes 12,150 Wh of GPU energy over 42.7 hours—nearly three times Mistral’s 4,211 Wh and seven times Gemma’s 1,654 Wh. However, when normalized by the number of resolved instances, Qwen is *more* efficient than Mistral (238 vs. 421 Wh per resolved instance) because its 5× higher resolve rate more than compensates for its higher absolute consumption.

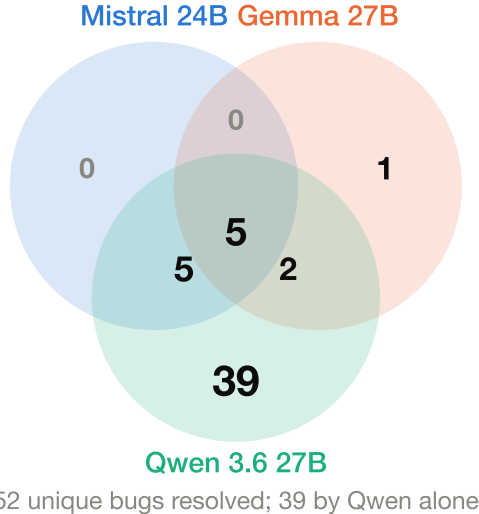


Figure 2: Venn diagram of resolved instances. Qwen resolves a strict superset of Mistral’s solutions and 39 bugs that no other model can fix. The union across all models is 52 unique bugs.

Table 3: Energy consumption and efficiency metrics.

	Mistral	Gemma	Qwen
Total runtime (h)	16.5	8.3	42.7
Avg GPU power (W)	255	198	285
Total GPU energy (Wh)	4,211	1,654	12,150
Net energy (Wh)	3,500	1,295	10,247
Energy / resolved (Wh)	421	207	238
Net energy / resolved	350	162	201

Gemma achieves the lowest energy per resolved instance (207 Wh) due to its short average instance duration (159 s) and low average GPU power (198 W). However, its low resolve rate (8.2%) means the vast majority of energy is spent on unsuccessful attempts.

4.4 Instance-Level Analysis

Figure 4 shows the distribution of per-instance durations, split by outcome. Across all three models, resolved instances complete faster than unresolved ones:

- **Mistral:** median 60 s (resolved) vs. 102 s (unresolved).
- **Gemma:** median 42 s vs. 111 s.
- **Qwen:** median 219 s vs. 425 s.

This pattern suggests that models “know when they’re stuck”—failed instances consume more iterations and time before the agent reaches the iteration limit or produces an incorrect patch.

Qwen’s durations are markedly longer than the other two models (median 219 s vs. ~ 60 s). This reflects deeper per-instance exploration: Qwen averages 16.4 tool calls over 18.6 turns per instance, compared to Mistral’s 23.9 tool calls over 18.9 turns. Despite fewer tool calls,

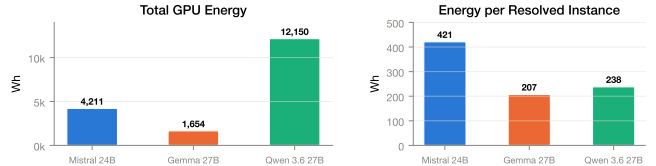


Figure 3: Left: total GPU energy per run. Right: energy per resolved instance. Despite consuming $\sim 3\times$ more total energy, Qwen achieves a lower cost per resolved bug than Mistral (238 vs. 421 Wh), because it resolves $\sim 5\times$ more instances.

Qwen spends more time because each interaction involves longer conversations and more computation-heavy terminal commands.

4.5 Tool Usage Patterns

Figure 5 shows the distribution of tool calls across models.

Mistral’s tool usage is dominated by file editing (68% of all calls), with only 29% terminal commands. This suggests a strategy of directly modifying source files with less exploratory testing. Qwen reverses this ratio (55% terminal, 36% file editor), indicating a more test-driven workflow: running existing tests, exploring the codebase with `grep` and `find`, and iteratively validating changes.

Table 4 provides a complete side-by-side summary of all collected metrics.

5 Discussion

Absolute vs. normalized energy. Our central finding is that total energy consumption is a misleading measure of efficiency for agentic workloads. Qwen consumes $\sim 3\times$ Mistral’s total energy but resolves $\sim 5\times$ more bugs, yielding a 44% lower energy cost per resolved instance. This parallels findings in other domains where effectiveness-normalized metrics (e.g., energy per correctly classified image) invert naïve rankings based on total consumption [?].

The cost of failure. Unresolved instances are not free—they consume GPU energy without producing value. Across all models, unresolved instances take longer (higher median duration) and consume more energy than resolved ones. Mistral’s 35% timeout rate represents substantial wasted energy: these instances consumed power for the full iteration budget without producing any patch. An early-exit heuristic that detects unproductive loops could significantly reduce waste.

Strategy and effectiveness. The tool-usage analysis reveals that higher resolve rates correlate with more terminal-heavy strategies. Qwen’s preference for running

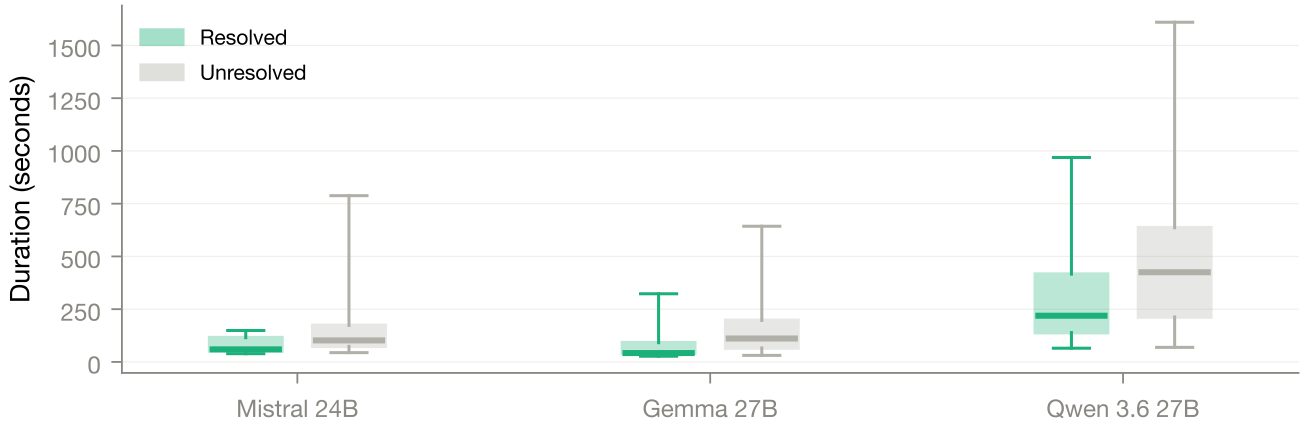


Figure 4: Per-instance duration distributions, split by resolved (green) and unresolved (gray). Resolved instances are consistently faster across all models. Qwen’s higher absolute durations reflect deeper exploration per instance (more tool calls, longer conversations), not idle time.

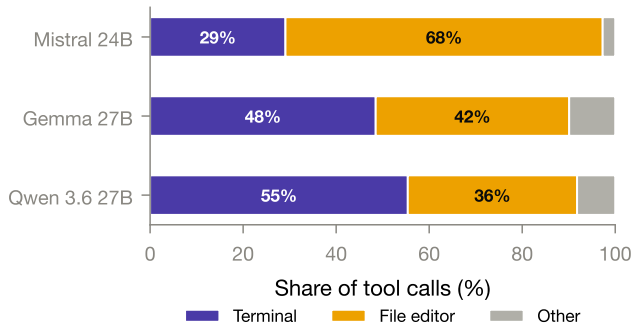


Figure 5: Tool-call distribution per model. Mistral is file-editor-dominant (68%), while Qwen and Gemma favor terminal commands (55% and 49%, respectively). Higher terminal usage correlates with Qwen’s deeper exploration and higher resolve rate.

tests and exploring the codebase via terminal commands (55% of tool calls) suggests a more effective debugging workflow than Mistral’s file-editing-dominant approach (68%). This aligns with software engineering best practices where understanding the failure mode through testing precedes fixing the code.

Model complementarity. Despite Qwen’s dominance, Gemma resolves one instance (`django-15061`) that Qwen does not, suggesting that model ensembles or cascading strategies could further improve coverage. The union of all three models’ resolved sets is 52 bugs—only one more than Qwen alone—indicating limited complementarity in this size class.

Limitations. Our study has several limitations:

- We evaluate only three models in a narrow parameter range (24–27 B). Smaller or larger models may show different energy–effectiveness trade-offs.

- SWE-bench Lite contains exclusively Python projects. Energy profiles may differ for other languages or task types.
- We measure GPU power only; CPU, memory, and network energy are excluded. GPU power dominates (> 80 % of system draw under load) but total system energy would be higher.
- Per-instance energy attribution uses averaged power over the instance duration, which may under- or over-count during periods of mixed GPU utilization (e.g., concurrent Docker operations).
- We run a single trial per model. Stochastic variation in LLM outputs means individual instance outcomes may differ across runs.

6 Conclusion

We present the first controlled energy benchmark of locally-hosted open-source LLMs on autonomous coding tasks. Our results demonstrate that: (1) task effectiveness varies dramatically across models of similar size, with Qwen 3.6 27B resolving 51/87 instances (58.6%) vs. $\leq 15.4\%$ for the other two models; (2) total energy consumption is a poor proxy for efficiency—the most energy-consuming model achieves the lowest cost per resolved bug; (3) distinct tool-usage strategies (terminal-heavy vs. editor-heavy) correlate with effectiveness; and (4) resolved instances complete faster than unresolved ones, suggesting that early-exit mechanisms could reduce energy waste.

These findings argue for reporting energy-per-resolved-task alongside raw resolve rates in future coding-agent benchmarks, and for developing energy-aware agent strategies that can terminate unproductive attempts early.

Table 4: Full comparison of all benchmark metrics across the three models. All models ran on the same hardware (A100 80 GB PCIe, TDP 300 W) with identical agent configuration (`n_limit=100`, `max_iterations=100`).

Category	Metric	Mistral Small 24B	Gemma 3 27B	Qwen 3.6 27B
Completion	Attempted	100	100	100
	Timed out	35	3	13
	Empty patch	1	15	0
	Evaluated	65	97	87
	Resolved	10 (15.4%)	8 (8.2%)	51 (58.6%)
Performance	Gen throughput (tok/s)	25.8	17.9	23.5
	Prompt throughput (tok/s)	158.3	188.4	433.0
	KV cache utilization (%)	7.9	5.6	6.8
	Total tokens (M)	21.1	29.0	30.2
Energy	Total runtime (h)	16.5	8.3	42.7
	Avg GPU power (W)	255	198	285
	Total GPU energy (Wh)	4,211	1,654	12,150
	Energy / resolved (Wh)	421	207	238
Behavior	Avg tool calls / inst.	23.9	15.8	16.4
	Terminal share (%)	29.1	48.5	55.4
	File editor share (%)	68.2	41.6	36.4
Instance duration (s)	Avg (all)	144	159	397
	Median resolved	60	42	219
	Median unresolved	102	111	425
	Max	788	643	1,610